Exp No :		Setting Up a Data Engineering Environment (Docker, Python, Jupyter, Git)
Date :		

Aim

To set up a complete local data engineering development environment using Docker, Python, Jupyter Notebook, and Git for reproducible pipeline development.

Algorithm

1. Install Docker Desktop / Docker Engine and verify installation.
2. Install Python (3.11+) and create an isolated virtual environment using venv or conda.
3. Create a docker-compose.yml file that defines a Jupyter Lab service with a mounted local volume.
4. Build and start the containers using docker compose up.
5. Initialize a Git repository, create a .gitignore for data and environment files, and make the first commit.
6. Verify the setup by launching Jupyter Lab in the browser and running a test Python script inside the container.
7. Push the repository to a remote (e.g., GitHub) for version control.

Program

[bash]

```
# 1. Verify Docker and Python installation
docker --version
docker compose version
python3 --version

# 2. Create project folder structure
mkdir de-lab && cd de-lab
mkdir -p notebooks data src
python3 -m venv venv
source venv/bin/activate
pip install pandas jupyterlab sqlalchemy psycopg2-binary
```

[yaml]

```
# docker-compose.yml
version: "3.9"
services:
  jupyter:
    image: jupyter/scipy-notebook:latest
    container_name: de_jupyter
    ports:
      - "8888:8888"
    volumes:
      - ./notebooks:/home/jovyan/work
      - ./data:/home/jovyan/data
    environment:
      - JUPYTER_TOKEN=delab123
```

[bash]

```
# 3. Start the environment
docker compose up -d

# 4. Initialize Git
git init
cat <<EOF > .gitignore
venv/
data/*.csv
*.pyc
__pycache__/
.env
EOF
git add .
git commit -m "Initial commit: DE environment setup"
```

Output

```
Docker version 27.3.1, build ce12230
Docker Compose version v2.29.7
Python 3.11.9
```

[+] Running 2/2

✓ Network de-lab_default Created

✓ Container de_jupyter Started

Jupyter Lab accessible at: http://127.0.0.1:8888/lab?token=delab123

Git repository initialized in /de-lab/.git/

[main (root-commit) 3f2a1c9] Initial commit: DE environment setup

4 files changed, 32 insertions(+)

Inference :

Viva voce Questions:

1. What is the difference between a Docker image and a Docker container?

2. Why is Git version control important in a data engineering workflow?

3. What is the purpose of a virtual environment in Python, and how does it differ from a Docker container?

4. Explain the role of a Dockerfile and what a typical instruction like FROM, RUN, or COPY does.

5. Why is Jupyter Notebook commonly used in data engineering and data science projects, and what are its limitations in production?

Result

Exp No :		ETL Pipeline using Python (Pandas) for CSV/JSON Data Processing
Date :		

Aim

To implement an Extract-Transform-Load (ETL) pipeline in Python using Pandas that reads raw CSV and JSON data, cleans and transforms it, and loads the result into a structured output file.

Algorithm

1. Extract: Read the source CSV file and the source JSON file into Pandas DataFrames.
2. Inspect the data for missing values, duplicates, and incorrect data types.
3. Transform: Clean missing values, cast columns to correct types, remove duplicates, and merge the CSV and JSON DataFrames on a common key.
4. Apply business transformations such as derived columns and aggregations.
5. Load: Write the cleaned, merged dataset to a Parquet/CSV file and log the pipeline run summary.
6. Validate the loaded output by re-reading it and checking row counts and schema.

Program

[python]

```
# etl_pipeline.py
import pandas as pd
import json
import logging
from datetime import datetime
 logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s | %(levelname)s | %(message)s")
log = logging.getLogger("etl")
 def extract(csv_path, json_path):
    log.info("Extracting data from CSV and JSON sources")
    df_csv = pd.read_csv(csv_path)
    with open(json_path) as f:
        df_json = pd.json_normalize(json.load(f))
    return df_csv, df_json
 def transform(df_csv, df_json):
```

```
    log.info("Transforming data")
    df_csv.drop_duplicates(inplace=True)
    df_csv["order_date"] = pd.to_datetime(df_csv["order_date"])
    df_csv["amount"] = df_csv["amount"].fillna(0).astype(float)
     df_json["customer_id"] = df_json["customer_id"].astype(int)
     merged = df_csv.merge(df_json, on="customer_id", how="left")
    merged["revenue_band"] = pd.cut(
        merged["amount"], bins=[0, 100, 500, float("inf")],
        labels=["low", "medium", "high"])
    return merged
def load(df, out_path):
    log.info(f"Loading {len(df)} rows to {out_path}")
    df.to_parquet(out_path, index=False)
def run_pipeline():
    start = datetime.now()
    df_csv, df_json = extract("data/orders.csv", "data/customers.json")
    merged = transform(df_csv, df_json)
    load(merged, "data/output/orders_enriched.parquet")
    log.info(f"Pipeline completed in {datetime.now() - start}")
    return merged
if __name__ == "__main__":
    result_df = run_pipeline()
    print(result_df.head())
```

Output

```
2026-09-10 10:02:11 | INFO | Extracting data from CSV and JSON sources
2026-09-10 10:02:11 | INFO | Transforming data
2026-09-10 10:02:11 | INFO | Loading 4821 rows to data/output/orders_enriched.parquet
2026-09-10 10:02:11 | INFO | Pipeline completed in 0:00:00.842311

   order_id  customer_id order_date  amount customer_name revenue_band
0      1001          204 2026-01-05  250.00     A. Sharma       medium
1      1002          198 2026-01-05   45.00     R. Verma          low
2      1003          204 2026-01-06  920.00     A. Sharma         high
3      1004          311 2026-01-06  180.50     K. Menon       medium
4      1005          276 2026-01-07   15.00     S. Iyer           low
```

Inference

Viva voce Questions

1. What are the three stages of an ETL pipeline, and what happens in each?

2. How does Pandas handle missing or null values, and what methods are used to treat them?

3. What is the difference between reading a CSV file and a JSON file in Pandas (read_csv vs read_json)?

4. How would you handle a very large dataset that does not fit into memory when using Pandas?

5. What is data normalization/transformation, and why is it a necessary step before loading data into a target system?

Result

Exp No :		ETL Pipeline using Python (Pandas) for CSV/JSON Data Processing
Date :		

Aim

To write and execute complex analytical SQL queries using window functions (ROW_NUMBER, RANK, LAG/LEAD) and Common Table Expressions (CTEs), including recursive CTEs, on a sales dataset.

Algorithm

1. Create the sample sales schema (orders, customers) and load sample data.
2. Write a query using ROW_NUMBER() to identify the most recent order per customer.
3. Write a query using RANK() and DENSE_RANK() to rank customers by total spend.
4. Write a query using LAG()/LEAD() to compute month-over-month revenue change.
5. Write a non-recursive CTE to pre-aggregate monthly revenue, then query it.
6. Write a recursive CTE to generate a date series / organizational hierarchy.
7. Execute all queries and record the result sets.

Program

[sql]

```
-- 1. Most recent order per customer (ROW_NUMBER)
SELECT customer_id, order_id, order_date, amount
FROM (
   SELECT *,
        ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date DESC) AS rn
   FROM orders
) t
WHERE rn = 1;


-- 2. Rank customers by total spend
SELECT customer_id,
     SUM(amount) AS total_spend,
     RANK()      OVER (ORDER BY SUM(amount) DESC) AS spend_rank,
     DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS dense_spend_rank
```

```
FROM orders
GROUP BY customer_id;

-- 3. Month-over-month revenue change (LAG)
WITH monthly_revenue AS (
   SELECT DATE_TRUNC('month', order_date) AS month,
        SUM(amount) AS revenue
   FROM orders
   GROUP BY 1
)
SELECT month, revenue,
      LAG(revenue)  OVER (ORDER BY month) AS prev_month_revenue,
      revenue - LAG(revenue) OVER (ORDER BY month) AS mom_change
FROM monthly_revenue
ORDER BY month;

-- 4. Recursive CTE: generate a calendar date series
WITH RECURSIVE date_series AS (
   SELECT DATE '2026-01-01' AS dt
   UNION ALL
   SELECT dt + INTERVAL '1 day'
   FROM date_series
   WHERE dt < DATE '2026-01-10'
)
SELECT dt FROM date_series;
```

Output

```
-- Query 1 result (sample)
 customer_id | order_id | order_date | amount
-------------+----------+------------+--------
         204 |     1003 | 2026-01-06 | 920.00
         198 |     1002 | 2026-01-05 |  45.00

-- Query 2 result (sample)
 customer_id | total_spend | spend_rank | dense_spend_rank
```

```
------------+-------------+------------+------------------
        204 |     1170.00 |          1 |                1
        311 |      180.50 |          2 |                2

-- Query 3 result (sample)
   month    | revenue  | prev_month_revenue | mom_change
------------+----------+--------------------+-----------
 2026-01-01 | 48210.00 |               NULL |       NULL
 2026-02-01 | 52340.00 |           48210.00 |    4130.00

-- Query 4 result: 10 rows, 2026-01-01 .. 2026-01-10
```

Inference

Viva voce Questions

1. What is a window function, and how does it differ from a GROUP BY aggregation?

2. Explain the difference between RANK(), DENSE_RANK(), and ROW_NUMBER().

3. What is a Common Table Expression (CTE), and why would you use one instead of a subquery?

4. Can you use a CTE recursively? Give an example scenario where a recursive CTE is useful.

Result.

Exp No :		SQL for Analytics — Query Optimization
Date :		

Aim

To analyze and optimize complex analytical SQL queries using EXPLAIN ANALYZE, indexing strategies, and query rewriting techniques.

Algorithm

1. Write a baseline analytical query joining large fact and dimension tables with a correlated subquery.
2. Run EXPLAIN ANALYZE on the baseline query and record the execution plan and timing.
3. Identify bottlenecks: sequential scans, nested loop joins, missing indexes.
4. Create appropriate indexes (B-tree on join/filter columns, composite indexes for common WHERE clauses).
5. Rewrite the correlated subquery as a JOIN / CTE to avoid repeated execution.
6. Re-run EXPLAIN ANALYZE and compare execution time and plan before vs after optimization.
7. Document the performance improvement.

Program

[sql]

```
-- Baseline (unoptimized) query: correlated subquery, no index
EXPLAIN ANALYZE
SELECT o.order_id, o.customer_id, o.amount,
      (SELECT c.customer_name FROM customers c
       WHERE c.customer_id = o.customer_id) AS customer_name
FROM orders o
WHERE o.amount > (SELECT AVG(amount) FROM orders);

-- Step 1: Add indexes on join/filter columns
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_amount ON orders(amount);
CREATE INDEX idx_customers_customer_id ON customers(customer_id);

-- Step 2: Rewrite subquery as JOIN + precomputed CTE
EXPLAIN ANALYZE
```

```
WITH avg_amount AS (
   SELECT AVG(amount) AS avg_amt FROM orders
)
SELECT o.order_id, o.customer_id, o.amount, c.customer_name
FROM orders o
JOIN customers c   ON c.customer_id = o.customer_id
CROSS JOIN avg_amount a
WHERE o.amount > a.avg_amt;
```

Output

```
-- BEFORE optimization
Seq Scan on orders o  (cost=0.00..184320.11 rows=48210 width=24)
  SubPlan 1 (per-row correlated subquery)
  SubPlan 2 (AVG recomputed for every row)
Planning Time: 0.412 ms
Execution Time: 3120.884 ms

-- AFTER optimization (indexes + JOIN rewrite)
Hash Join  (cost=1.11..2814.30 rows=16070 width=24)
  -> Index Scan using idx_orders_amount on orders o
  -> Hash  (Index Scan on idx_customers_customer_id)
Planning Time: 0.298 ms
Execution Time: 96.217 ms
```

Inference

Viva voce Questions

1. What is a query execution plan, and how do you use EXPLAIN/EXPLAIN ANALYZE to read it?

2. How does indexing improve query performance, and what are the trade-offs of having too many indexes?

3. What is the difference between a clustered and a non-clustered index?

4. Why can using SELECT * instead of specific column names hurt performance?

5. What is query predicate pushdown, and how does

Result

Exp No :		Data Warehouse Schema Design and ETL Load
Date :		

Aim

To design a star-schema data warehouse (fact and dimension tables) and implement an ETL process to load transformed data into it.

Algorithm

1. Identify the business process (sales) and grain of the fact table (one row per order line).
2. Design dimension tables: dim_customer, dim_product, dim_date (with SCD Type 2 for dim_customer).
3. Design the fact table: fact_sales with foreign keys to each dimension and additive measures.
4. Create the schema in the warehouse (PostgreSQL) using DDL.
5. Implement an ETL script that extracts source data, looks up/generates surrogate keys, and performs an upsert (incremental load) into dimension and fact tables.
6. Validate the load by running a sample aggregation query against the star schema.

Program

[sql]

```
-- Dimension: Customer (SCD Type 2)
CREATE TABLE dim_customer (
   customer_key    SERIAL PRIMARY KEY,
   customer_id     INT NOT NULL,
   customer_name   VARCHAR(100),
   city            VARCHAR(50),
   valid_from      DATE NOT NULL,
   valid_to        DATE,
   is_current      BOOLEAN DEFAULT TRUE
);

CREATE TABLE dim_product (
   product_key   SERIAL PRIMARY KEY,
   product_id    INT NOT NULL,
   product_name  VARCHAR(100),
```

```
    category      VARCHAR(50)
);

CREATE TABLE dim_date (
    date_key   INT PRIMARY KEY,   -- YYYYMMDD
    full_date  DATE,
    year INT, quarter INT, month INT, day INT, weekday VARCHAR(10)
);

CREATE TABLE fact_sales (
    sales_key     SERIAL PRIMARY KEY,
    date_key      INT REFERENCES dim_date(date_key),
    customer_key  INT REFERENCES dim_customer(customer_key),
    product_key   INT REFERENCES dim_product(product_key),
    quantity      INT,
    amount        NUMERIC(12,2)
);
```

[python]

```
# warehouse_etl.py
import pandas as pd
from sqlalchemy import create_engine, text
 engine = create_engine("postgresql://dwuser:dwpass@localhost:5432/salesdw")
 def load_dim_date(df_dates):
    df_dates.to_sql("dim_date", engine, if_exists="append", index=False,
                    method="multi")
 def upsert_dim_customer(df_customers):
    with engine.begin() as conn:
        for _, row in df_customers.iterrows():
            conn.execute(text("""
                UPDATE dim_customer SET is_current = FALSE, valid_to = CURRENT_DATE
                WHERE customer_id = :cid AND is_current = TRUE
                  AND (customer_name != :name OR city != :city)
            """), {"cid": row.customer_id, "name": row.customer_name, "city": row.city})
            conn.execute(text("""
```

```
            INSERT INTO dim_customer (customer_id, customer_name, city, valid_from, is_current)
            SELECT :cid, :name, :city, CURRENT_DATE, TRUE
            WHERE NOT EXISTS (
              SELECT 1 FROM dim_customer
              WHERE customer_id = :cid AND is_current = TRUE)
        """), {"cid": row.customer_id, "name": row.customer_name, "city": row.city})

def load_fact_sales(df_fact):
    df_fact.to_sql("fact_sales", engine, if_exists="append", index=False)

print("Warehouse load complete.")
```

Output

```
CREATE TABLE (dim_customer, dim_product, dim_date, fact_sales)
Loaded 3650 rows into dim_date
Upserted 512 rows into dim_customer (18 new SCD2 versions)
Loaded 4821 rows into fact_sales
Warehouse load complete.

-- Validation query
SELECT c.city, SUM(f.amount) AS revenue
FROM fact_sales f JOIN dim_customer c ON f.customer_key = c.customer_key
WHERE c.is_current GROUP BY c.city ORDER BY revenue DESC LIMIT 3;

  city    | revenue
----------+----------
 Chennai   | 812400.50
 Bengaluru | 745120.00
 Mumbai    | 690310.25
```

Inference

Viva voce Questions

1. What is the difference between a star schema and a snowflake schema?

2. What are fact tables and dimension tables, and how do they relate to each other?

3. What is a Slowly Changing Dimension (SCD), and explain the difference between SCD Type 1 and Type 2.

4. What is the difference between OLTP and OLAP systems?

5. What is the difference between a full load and an incremental load in an ETL process?

Result

Exp No :		Real-Time Data Pipeline using Kafka for Streaming Data
Date :		

Aim

To implement a real-time streaming data pipeline using Apache Kafka, with a Python producer generating events and a consumer processing them in real time.

Algorithm

1. Start a Kafka broker (and Zookeeper/KRaft) using Docker Compose.
2. Create a Kafka topic for streaming order events.
3. Implement a Producer script that generates simulated order events and publishes them to the topic.
4. Implement a Consumer script that subscribes to the topic, deserializes messages, and performs a real-time windowed aggregation (e.g., rolling revenue per minute).
5. Run the producer and consumer concurrently and observe real-time processing.
6. Verify message delivery and processed aggregates.

Program

[yaml]

```
# docker-compose.yml (Kafka in KRaft mode)
services:
  kafka:
    image: bitnami/kafka:3.7
    ports: ["9092:9092"]
    environment:
      - KAFKA_CFG_NODE_ID=1
      - KAFKA_CFG_PROCESS_ROLES=broker,controller
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
```

[python]

```
# producer.py
```

```
import json, time, random
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode())

while True:
    event = {
        "order_id": random.randint(10000, 99999),
        "customer_id": random.randint(100, 400),
        "amount": round(random.uniform(10, 1000), 2),
        "ts": time.time()
    }
    producer.send("orders_stream", value=event)
    print("Produced:", event)
    time.sleep(0.5)
```

[python]

```
# consumer.py
import json
from collections import defaultdict
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    "orders_stream",
    bootstrap_servers="localhost:9092",
    value_deserializer=lambda v: json.loads(v.decode()),
    auto_offset_reset="latest")

window_revenue = defaultdict(float)

for msg in consumer:
    event = msg.value
    minute_bucket = int(event["ts"] // 60)
```

```
    window_revenue[minute_bucket] += event["amount"]
    print(f"Consumed order {event['order_id']} | "
          f"Running revenue this minute: {window_revenue[minute_bucket]:.2f}")
```

Output

Producer:

Produced: {'order_id': 47213, 'customer_id': 204, 'amount': 512.44, 'ts': 1757500001.2}

Produced: {'order_id': 88192, 'customer_id': 311, 'amount': 88.10, 'ts': 1757500001.7}

Consumer:

Consumed order 47213 | Running revenue this minute: 512.44

Consumed order 88192 | Running revenue this minute: 600.54

Consumed order 19042 | Running revenue this minute: 1043.09

Inference

Viva voce Questions

1. What are the core components of Kafka's architecture (producer, consumer, broker, topic, partition)?

2. Why does Kafka use partitions within a topic, and how does this enable scalability?

3. What is a consumer group, and how does it enable parallel processing of messages?

4. What is the role of Zookeeper (or KRaft) in a Kafka cluster?

5. How does Kafka ensure fault tolerance and message durability through replication?

Result

Exp No :		Orchestrating Data Pipelines using Apache Airflow
Date :		

Aim

To orchestrate the ETL and warehouse-load pipeline as a Directed Acyclic Graph (DAG) in Apache Airflow, with scheduling, retries, and task dependencies.

Algorithm

1. Set up Airflow using the official Docker Compose file (webserver, scheduler, metadata DB).
2. Define a DAG with a daily schedule interval.
3. Define tasks: extract_data, transform_data, load_warehouse, notify, using PythonOperator.
4. Set task dependencies so extract -> transform -> load -> notify.
5. Configure retries, retry delay, and failure alerting on each task.
6. Trigger the DAG manually from the Airflow UI/CLI and monitor execution via the Graph view.
7. Verify successful completion of all tasks and check logs.

Program

[python]

```
# dags/sales_etl_dag.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    "owner": "data_eng",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
}

def extract_data(**kwargs):
    from src.etl_pipeline import extract
    extract("data/orders.csv", "data/customers.json")

def transform_data(**kwargs):
```

```
    from src.etl_pipeline import transform
    transform(**kwargs)

def load_warehouse(**kwargs):
    from src.warehouse_etl import load_fact_sales
    load_fact_sales(**kwargs)

def notify(**kwargs):
    print("Pipeline run completed successfully.")

with DAG(
    dag_id="sales_etl_pipeline",
    default_args=default_args,
    schedule_interval="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["sales", "etl"],
) as dag:

    t1 = PythonOperator(task_id="extract_data", python_callable=extract_data)
    t2 = PythonOperator(task_id="transform_data", python_callable=transform_data)
    t3 = PythonOperator(task_id="load_warehouse", python_callable=load_warehouse)
    t4 = PythonOperator(task_id="notify", python_callable=notify)

    t1 >> t2 >> t3 >> t4
```

[bash]

```
# Start Airflow and trigger the DAG
docker compose -f airflow-docker-compose.yml up -d
airflow dags trigger sales_etl_pipeline
airflow tasks list sales_etl_pipeline --tree
```

Output

```
DAG: sales_etl_pipeline
```

extract_data   -> success (0:00:04)

transform_data -> success (0:00:07)

load_warehouse -> success (0:00:03)

notify   -> success (0:00:01)

Run: manual__2026-09-10T10:15:00+00:00 State: success

Pipeline run completed successfully.

Inference

Viva voce Questions

1. What is a DAG in Airflow, and why is it required to be acyclic?

2. What is the difference between an Operator, a Task, and a TaskInstance in Airflow?

3. How does Airflow's scheduler decide when to trigger a DAG run?

4. What is the purpose of XComs in Airflow, and when would you use them?

5. What is the difference between task dependencies set using >> (bitshift operators) and trigger rules like ALL_SUCCESS or ONE_FAILED?

Result

Exp No :		Interactive Dashboard using Apache Superset for Business Insights
Date :		

Aim

To build an interactive business-intelligence dashboard on top of the data warehouse using Apache Superset, visualizing key sales metrics.

Algorithm

1. Deploy Apache Superset using Docker Compose.
2. Connect Superset to the PostgreSQL data warehouse (salesdw) as a new database source.
3. Create a virtual dataset (SQL Lab query) joining fact_sales with dim_customer, dim_product, and dim_date.
4. Create charts: revenue trend over time (line chart), revenue by city (bar chart), top products (table), and revenue by category (pie chart).
5. Assemble the charts into a single dashboard with filters (date range, city).
6. Publish the dashboard and verify interactivity (filtering, drill-down).

Program

[bash]

```
# Deploy Superset
git clone https://github.com/apache/superset.git
cd superset
docker compose -f docker-compose-non-dev.yml up -d

# Create admin user
docker exec -it superset_app superset fab create-admin \
  --username admin --password admin \
  --firstname Admin --lastname User --email admin@delab.local
docker exec -it superset_app superset db upgrade
docker exec -it superset_app superset init
```

[sql]

```
-- Virtual dataset used for the dashboard (SQL Lab)
```

```
SELECT d.full_date, c.city, p.category, p.product_name,
       f.quantity, f.amount
FROM fact_sales f
JOIN dim_date d     ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key AND c.is_current
JOIN dim_product p  ON f.product_key = p.product_key;
```

Output

```
Superset running at http://localhost:8088
Database connection 'salesdw' added successfully
Dataset 'sales_fact_view' created (4821 rows)

Dashboard: 'Sales Business Insights'
  - Revenue Trend (Line): Jan-Sep 2026, upward trend, peak in July
  - Revenue by City (Bar): Chennai leads at 812,400.50
  - Top Products (Table): 10 rows, top item 'Wireless Mouse'
  - Revenue by Category (Pie): Electronics 46%, Apparel 28%, Other 26%
Filters: Date Range, City  -> verified: chart updates on filter change
```

Inference

Viva voce Questions

1. What is the role of a semantic/data layer in Superset, and why is a database connection required before creating charts?

2. What is the difference between a Superset dataset and a physical database table?

3. How do filters and dashboard-level cross-filtering improve the interactivity of a dashboard?

4. What is the difference between a virtual dataset (SQL-based) and a physical dataset in Superset?

5. Why is caching important in a BI tool like Superset, and how does it affect dashboard performance?

Result

Exp No :		Containerizing and Deploying the Data Application using Docker
Date :		

Aim

To containerize the complete data application (ETL scripts, dependencies) using Docker and deploy the full multi-service pipeline with Docker Compose.

Algorithm

1. Write a Dockerfile for the Python ETL application, installing dependencies from requirements.txt.
2. Build the Docker image and tag it appropriately.
3. Write a master docker-compose.yml combining the ETL app, PostgreSQL warehouse, Kafka, Airflow, and Superset services.
4. Define service dependencies (depends_on) and shared networks/volumes.
5. Bring up the entire stack with a single command and verify each service's health.
6. Push the built image to a container registry (e.g., Docker Hub) for deployment.

Program

[dockerfile]

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
COPY data/ ./data/
ENTRYPOINT ["python", "src/etl_pipeline.py"]
```

[yaml]

```
# docker-compose.yml (full stack)
services:
  etl_app:
    build: .
    image: delab/etl-app:1.0
```

```
    depends_on: [warehouse_db]
    networks: [delab_net]

  warehouse_db:
    image: postgres:16
    environment:
      POSTGRES_DB: salesdw
      POSTGRES_USER: dwuser
      POSTGRES_PASSWORD: dwpass
    volumes: ["pgdata:/var/lib/postgresql/data"]
    networks: [delab_net]

  kafka:
    image: bitnami/kafka:3.7
    networks: [delab_net]

  superset:
    image: apache/superset:latest
    ports: ["8088:8088"]
    depends_on: [warehouse_db]
    networks: [delab_net]

networks:
  delab_net:
volumes:
  pgdata:
```

[bash]

```
# Build and deploy
docker build -t delab/etl-app:1.0 .
docker compose up -d
docker compose ps

# Push to registry
docker tag delab/etl-app:1.0 myrepo/delab-etl-app:1.0
```

docker push myrepo/delab-etl-app:1.0

Output

Successfully built 7c2a91e4f3ab

Successfully tagged delab/etl-app:1.0

NAME STATUS PORTS

etl_app Up (healthy) -

warehouse_db Up (healthy) 5432/tcp

kafka Up (healthy) 9092/tcp

superset Up (healthy) 0.0.0.0:8088->8088/tcp

The push refers to repository [myrepo/delab-etl-app]

1.0: digest: sha256:9af3c1... size: 1571

Inference

Viva voce Questions

1. What is the purpose of a docker-compose.yml file, and how does it differ from running individual docker run commands?

2. What is the difference between Docker volumes and bind mounts, and why are they important for data persistence?

3. How does Docker networking allow multiple containers (e.g., an app and a database) to communicate with each other?

4. What is a multi-stage Docker build, and why is it used to reduce image size?

5. What is the difference between exposing a port with EXPOSE in a Dockerfile and publishing it with -p during docker run?

Result

Exp No :		End-to-End Capstone Project: Complete Data Pipeline from Ingestion to Visualization
Date :		

Aim

To integrate all previous components — environment setup, ETL, SQL analytics, warehouse modeling, streaming, orchestration, dashboarding, and containerization — into a single end-to-end data pipeline, from raw data ingestion to a business-facing visualization dashboard.

Algorithm

1. Ingest raw batch data (CSV/JSON) and real-time streaming data (Kafka) as the two data sources.
2. Run the Pandas ETL pipeline to clean and enrich the batch data.
3. Load both batch and stream-aggregated data into the star-schema data warehouse.
4. Orchestrate the entire batch flow (extract -> transform -> load) as an Airflow DAG scheduled daily.
5. Run complex analytical SQL (window functions, CTEs) on the warehouse for reporting views.
6. Containerize all components (ETL app, Kafka, warehouse, Airflow, Superset) with Docker Compose for one-command deployment.
7. Build a final Superset dashboard consuming the warehouse to present end-to-end business insights.
8. Validate the pipeline end-to-end: trigger ingestion, confirm DAG success, confirm dashboard reflects new data.

Program

[text]

Capstone Architecture

```
----------------------
 [CSV/JSON files]      [Kafka producer: live orders]
      |                    |
      v                    v
 Pandas ETL (extract/transform)   Kafka Consumer (windowed agg)
      |                    |
      +------------+-------------+
                   v
      PostgreSQL Data Warehouse (star schema)
                   |
```

```
      +-------------+--------------+
      v                            v
  SQL Analytics views          Apache Superset dashboard
      ^
      |
  Orchestrated end-to-end by Apache Airflow DAG
  All services containerized via Docker Compose
```

[python]

```
# dags/capstone_pipeline_dag.py  (ties together experiments 2, 5, 6, 8)
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {"owner": "data_eng", "retries": 2,
                "retry_delay": timedelta(minutes=3)}

with DAG("capstone_end_to_end", default_args=default_args,
         schedule_interval="@daily", start_date=datetime(2026, 1, 1),
         catchup=False) as dag:

    ingest_batch = PythonOperator(
        task_id="ingest_batch",
        python_callable=lambda: __import__('src.etl_pipeline', fromlist=['run_pipeline']).run_pipeline())

    load_dw = PythonOperator(
        task_id="load_warehouse",
        python_callable=lambda: __import__('src.warehouse_etl', fromlist=['load_fact_sales']))

    refresh_views = BashOperator(
        task_id="refresh_sql_views",
        bash_command="psql -h warehouse_db -U dwuser -d salesdw -f sql/refresh_views.sql")

    refresh_dashboard = BashOperator(
```

```
        task_id="refresh_superset_cache",
        bash_command="docker exec superset_app superset refresh-cache")

    ingest_batch >> load_dw >> refresh_views >> refresh_dashboard
```

Output

```
DAG: capstone_end_to_end
ingest_batch          -> success (0:00:05)
load_warehouse        -> success (0:00:04)
refresh_sql_views     -> success (0:00:02)
refresh_superset_cache-> success (0:00:01)

docker compose ps
All 5 services: Up (healthy)

Superset Dashboard 'End-to-End Sales Insights':
  Total Revenue (YTD): Rs. 24,81,900.75
  Live streaming revenue (last hour): Rs. 12,340.20
  Top city: Chennai | Top category: Electronics
Dashboard reflects data ingested in the latest DAG run -> validation PASSED
```

Inference

Result